

VERTICAL AI APPLICATIONS · ENTERPRISE

KNACKLABS

We ship governed AI apps enterprises trust. **Fast.**

KnackLabs builds vertical AI applications — tender discovery, customer service, and the workflows after them — for enterprises. Every one runs on **Gantry, the engine we built**: a proprietary governed agent runtime. Each vertical we ship makes the next one faster and cheaper.

Paid before launch

Two enterprises committed money before we launched. That is the strongest signal a pre-launch company can give: customers who paid before they saw a finished product.

Customer	Vertical	Channel	Status
Manipal Group	Tender discovery	Microsoft Teams	IN TESTING
Bombay Sweet House	Customer service	WhatsApp · Interakt	IN BUILD
Additional tender buyers	Tender discovery	Multiple	IN CONVERSATION

The Manipal tender agent is built on Gantry, with the Teams integration in final testing. The Bombay Sweet House customer-service agent is in build on WhatsApp. Both are targeted to go live this summer. More tender buyers are in active conversations (interest shown, not yet signed).

Two paid, signed build engagements, both inbound, both before the product was finished.

You cannot write an agent's job description before it starts working

Enterprises want AI agents. But an agent is not software you can specify in advance. You discover what it should do by running it. That breaks the normal way enterprises buy and build software, and it is why so much agent ambition is stuck in pilots.

Agents are labor, not code. Give two agents the same instructions and different memory and they behave differently. No two are identical, so a static spec can never fully hold

them.

Without a security perimeter and an audit trail, no enterprise will put an agent near real work. So most agent projects stall before production: impressive demo, then nothing a CISO will sign off.

The result is enormous demand and almost nothing live. That gap is the opportunity.

03 / WHAT WE DO

We ship the governed vertical app that gets past the pilot

KnackLabs builds vertical, domain-specific AI applications, governed, auditable, and shipped fast. We start with the workflow an enterprise already pays people to do, and we deliver an agent that does it inside the tools they already use.

First build for a new client is bespoke. Then the vertical becomes a product. The second customer in a vertical gets most of it off the shelf, because the hard parts already exist.

We meet the enterprise where they work: Teams, WhatsApp, Slack, web. Gantry is embedded as a library inside their stack through our SDK, not a portal they have to migrate onto.

04 / WHY WE WIN

Gantry: the governed engine a dev shop will not build for one client

Every app we ship runs on Gantry, our proprietary agent runtime. An enterprise could try to build this themselves; it would take a team a year and a security review to get right, for one workflow. We bring it on day one, and reuse it on every vertical after. That is why our apps are trustworthy enough for an enterprise, and why each new build is cheap.

The core idea: an agent holds no standing power. It earns capabilities one at a time. A capability is composed from a source, a skill, an MCP server, or a tool, and the agent can

only *request* it. The conversation's human admin is the only one who can approve; the agent can never grant itself anything; every grant and every action is written to an audit trail; and the jobs an agent schedules carry their own scoped permissions. This is the answer to the problem above. You put a nondeterministic worker into a real workflow by granting trust narrowly, having a human approve it, and auditing all of it. The industry agrees this is required; most tools stop at allowing or blocking a whole server. We do it per capability.

Auditable by construction

Every action is logged and traceable. This is the precondition for putting an agent anywhere near work that matters, regulated or not.

A security perimeter per conversation

Every tool call passes a who-and-what gate. Agents never hold raw keys; access is granted and revoked the way you manage an employee's permissions, and agents cannot approve themselves.

A governed capability lifecycle

Agents cannot silently expand their own powers. New tools move through request, review, audit, and versioned activation.

Scoped memory with a dreaming lifecycle

Boundaries are enforced at the data layer, and a background process promotes raw conversation into vetted, durable memory, surfacing contradictions instead of overwriting them.

A provider-neutral model gateway IN PROGRESS

Not locked to one model vendor. This is also the precondition for running our own distilled models later (see the moat).

One agent, every channel

Slack and Telegram are live today; Teams and WhatsApp are in final integration for our first two customers; voice is next. The same perimeter, memory, and audit on every surface.

Each vertical makes the next one cheaper

A custom AI build is a project. A vertical that productizes is a company. The difference is what happens on the *second* deployment.

The first build in a vertical is bespoke: real engineering, services margin. Every build after it reuses Gantry plus the vertical's template, so the one-time build fee shrinks while recurring revenue grows. Margin bends from agency toward software.

LAYER 1 • ONE-TIME

Build fee

The bespoke first build per client. Shrinks each time the vertical is resold.

LAYER 2 • RECURRING

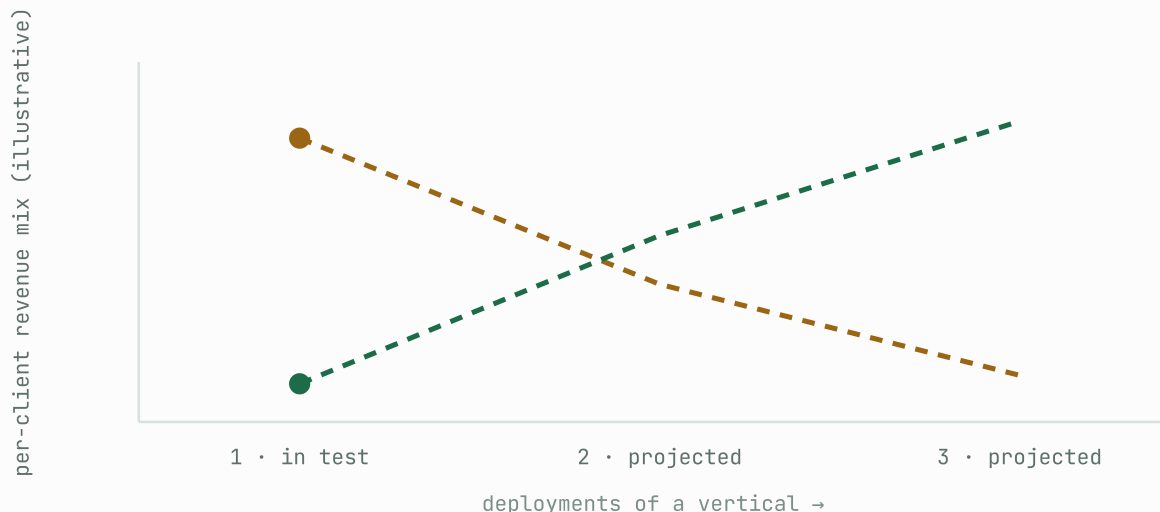
Platform license

Annual, predictable. The product revenue that compounds.

LAYER 3 • VALUE

Usage / outcome

For tender: per bid prepared or per tender won. Priced to the customer's ROI.



— One-time build fee (falls) — Recurring license + usage (grows)

— dot = actual (deployment 1), dashed = projected

Illustrative model, not historical data, and a revenue mix rather than absolute dollars. Deployment 1 is our flagship, in test today; deployments 2 and 3 are projected as the vertical productizes.

The Expert Learning Model ON THE ROADMAP

This is where the conversations compound. Today every vertical runs on frontier models, which is correct: they are the most capable, and every conversation they handle becomes labeled training data for that domain. Once a vertical has volume, we fine-tune a small, specialized model on that data, an Expert Learning Model, tuned per domain and

per customer. The ELM serves everyday inference cheaply; the frontier model is called only for the hard cases and to correct the ELM when it is wrong, and each correction becomes the next round of training.

As agents take on more work, model inference becomes the largest cost of delivering a vertical, and that cost should not be hostage to one vendor's pricing. An ELM that is excellent at one domain, or one customer's version of it, and cheap to run, is a margin and quality advantage no horizontal platform can hand a customer. The provider-neutral model gateway we are building now is the precondition: it is what lets us route between a frontier model and our own ELM without rewriting an app. This is the plan, not a shipped capability, and it is why provider-neutrality comes first.

We are proving this now: one vertical, first paid client in test, more buyers in conversation, a second vertical in build. It becomes proven the day tender client #2 signs.

06 / WHERE WE ARE

The engine is real, and we grade ourselves

Status as of May 2026, measured from the codebase. The runtime is substantial and tested: 281 test files, 17 architecture decision records, and an architecture layering rule enforced in CI, across more than 260,000 lines of TypeScript. The point is not the line count; it is that the engine is real, tested, and disciplined. We keep ourselves honest about what is and is not done.

Capability	Status	Notes
Core runtime · capability permissions · human-only approval · audit	BUILT	Capabilities composed from skills, MCP servers, and tools; conversation-admin approval only; no agent self-approval; every grant and action audited; jobs carry scoped permissions. Runtime-enforced.

Capability	Status	Notes
Slack & Telegram · async jobs · SDK + Control API · MCP / browser / webhooks	BUILT	Native surfaces, durable jobs, per-tool allowlists, HMAC-signed delivery, full audit.
Semantic (vector) memory retrieval	IN PROGRESS	Embeddings and the pgvector column are present; lexical recall is live today, vector ranking shipping now.
Microsoft Teams transport	IN PROGRESS	Normalization and approvals done; Bot Framework transport in final testing with Manipal.
Provider-neutral execution · personal → enterprise migration	IN PROGRESS	Clean adapter seam, one live provider (Anthropic) today; second provider next.
Voice channel	NEXT	Planned first-class channel; the surface that most defines a labor replacer.

07 / ROADMAP

The last mile, and the playbook that repeats

- SUMMER 2026 – first verticals live + foundation closed
 - Manipal tender agent live on Teams; Bombay Sweet House customer service live on WhatsApp (target).
 - Turn on semantic (vector) memory retrieval — the staged pgvector path.
 - Complete the personal-runtime → enterprise domain migration to parity.
 - Clean packaging to [@knacklabs/gantry](#) — private, enterprise metadata.

● BY Q4 2026 – multi-channel, multi-provider, hardened

- Prove a second model provider live through the existing adapter seam.
- Sign tender client #2 — the moment "proving repeatability" becomes "proven."
- Hardening: architecture docs, second owners to reduce key-person risk, observability.

● THE PLAYBOOK – ongoing

- Ship a vertical, productize it, resell it. Each one ships the next one faster.
- Begin per-vertical Expert Learning Models (fine-tuned domain and customer models) once a vertical has volume.
- Add Voice, then Capcom fleet control (below).

The shape of the company: services revenue today, bending toward product revenue as each vertical productizes, with a data moat that compounds per vertical.

08 / THE VISION, EARNED

Every application's core becomes an agent

The deterministic shell of an app (auth, user management, dashboards) becomes scaffolding. The real work gets done by an agent connecting to the enterprise's tools. Our SDK is how that app gets built, and Gantry is what makes the agent safe enough to ship.

WHY VOICE MATTERS

A labor replacer that cannot talk is not, fully, a labor replacer. Support, collections, scheduling, and front-office work live on the phone. Voice, governed by the same perimeter, memory, and audit as every other channel, is what lets an agent stand in for a role defined by conversation.

CAPCOM

One screen for every Gantry agent an enterprise runs.

Once an enterprise runs more than one agent, the next need is obvious: one view of health, activity, the full audit trail, cost, and the ability to intervene. Gantry holds and fuels each agent; **Capcom is mission control for the fleet.**

This is where fleet-level governance and switching cost compound, the eventual, earned answer to the multi-agent control plane we deliberately do not claim today. We earn it one vertical at a time.

KNACKLABS

Chimps At Work Studios Pvt Ltd · KnackLabs · Confidential — for discussion under NDA · v0.3, May 2026